lab1

1 链接器脚本与内核内存布局

1.1 运行 `make` 构建内核。

```
root@zju-os /z/code (lab1)# make
make -C kernel clean
make[1]: Entering directory '/zju-os/code/kernel'
make -C lib clean
make[2]: Entering directory '/zju-os/code/kernel/lib'
rm -f *.o *.d
make[2]: Leaving directory '/zju-os/code/kernel/lib'
make -C arch/riscv clean
make[2]: Entering directory '/zju-os/code/kernel/arch/riscv'
make -C kernel clean
make[3]: Entering directory '/zju-os/code/kernel/arch/riscv/kernel'
rm -f *.o *.d
make[3]: Leaving directory '/zju-os/code/kernel/arch/riscv/kernel'
```

1.2 用 VSCode 打开 `kernel/vmlinux`（已默认绑定到 ElfPreview 插件），查看它的内容。

如下图所示，包含分区、标志等

kernel > vmlinux

vmlinux Debug Symbols

File Information

File Type:	EXEC	Entry Point:	0x80200000
Machine Architecture:	RISCV	ELF Version:	1
Class:	ELF64	Symbols:	163
Endianness:	Big Endian	Sections:	19
OS ABI:	UNIX - System V		

Sections | Program Headers | Symbols

Section Headers

Name	Type	Address	Size
	SHT_NULL	0x00000000	0
.text	SHT_PROGBITS	0x80200000	8280

Sections | Program Headers | Symbols

Symbols

Filter name... 163 of 163 symbols

Value	Size	Type	Bind	Vis	Name
0x00000000	0	NOTYPE	LOCAL	DEFAULT	
0x80200000	0	SECTION	LOCAL	PROTEC…	
0x80203000	0	SECTION	LOCAL	PROTEC…	
0x802034f8	0	SECTION	LOCAL	PROTEC…	
0x80204000	0	SECTION	LOCAL	PROTEC…	
0x80204000	0	SECTION	LOCAL	PROTEC…	
0x00000000	0	SECTION	LOCAL	PROTEC…	
0x00000000	0	SECTION	LOCAL	PROTEC…	
0x00000000	0	SECTION	LOCAL	PROTEC…	
0x00000000	0	SECTION	LOCAL	PROTEC…	

2 OpenSBI 调试

2.1 按 Lab0 中的步骤启动 QEMU 和 GDB 调试。

使用make debug和make gdb

2.2 GDB 在 OpenSBI 跳转到的地址（Next Addr）处设置断点，查看此时：

2.2.1 `sp` 寄存器的值是多少？

先打断点：break *0x80200000，然后continue
之后info registers sp

```
head.S
B+>    11      tail start_kernel

       13      .section .bss.stack
       14      .space PGSIZE

B+>0x80200000 <_stext>            j       0x8020000c <start_ker
   0x80200004 <_traps>            jal     0x80200748 <trap_hand
   0x80200008 <_traps+4>          sret
   0x8020000c <start_kernel>      addi    sp,sp,-112
   0x80200010 <start_kernel+4>    sd      ra,104(sp)
   0x80200014 <start_kernel+8>    sd      s0,96(sp)
   0x80200018 <start_kernel+12>   auipc   a0,0x3
   0x8020001c <start_kernel+16>   addi    a0,a0,-24
   0x80200020 <start_kernel+20>   jal     0x80200148 <printk>

remote Thread 1.1 (src) In: _stext          L11   PC: 0x80200000
Breakpoint 1 at 0x80200000: file head.S, line 11.
(gdb) c
Continuing.

Breakpoint 1, _stext () at head.S:11
(gdb) info registers sp
sp             0x80046eb0       0x80046eb0
```

2.2.2 这属于哪个区域，该区域各个特权级的权限是什么？

如图所示，数值为0x80046eb0

`vmlinux.lds` 明确划分了 **OpenSBI 占用区域** 和 **内核可用区域**

链接脚本参数	数值 / 公式	含义说明
PHY_START	0x80000000	物理内存起始地址（RISC-V virt 平台默认内核加载基地址）
OPENSBI_SIZE	0x200000（即 2MB）	OpenSBI（M 模式固件）占用的内存大小
BASE_ADDR	PHY_START + OPENSBI_SIZE = 0x80200000	内核内存区域的起始地址（OpenSBI 结束后，内核代码 / 数据的开始）
ram 区域（内核用）	起始 0x80200000，长度 126MB	内核专属内存（PHY_SIZE=128MB 减去 OPENSBI_SIZE=2MB）
OpenSBI 区域	0x80000000 ~ 0x80200000	OpenSBI 固件的代码、数据、栈所在区域（内核不可直接使用）

所以0x80046eb0位于OpenSBI占用的物理内存的范围中

各个特权级权限：M模式（R/W/X（部分子区域））、S模式（无权限）、U模式（无权限）

2.3 用 VSCode 打开 `kernel/arch/riscv/boot/Image`（已默认绑定到 Hex Editor 插件），然后拉到最底下，观察 Hex Editor 左侧显示的文件偏移地址，它的大小是多少？

```
00003A50  02 58 C1 44 C8 44 C9 44 0E 00 00 00 00 00 00 00
00003A60  28 00 00 00 6C 05 00 00 84 E4 FF FF 70 00 00 00
00003A70  00 44 0E 20 50 81 02 88 04 89 06 92 08 02 48 C1
00003A80  44 C8 44 C9 44 D2 44 0E 00 00 00 00 1C 00 00 00
00003A90  98 05 00 00 C8 E4 FF FF 64 00 00 00 00 44 0E 50
00003AA0  44 81 02 02 54 C1 44 0E 00 00 00 00 18 00 00 00
00003AB0  B8 05 00 00 0C E5 FF FF 24 00 00 00 00 44 0E 10
00003AC0  44 81 02 54 C1 44 0E 00 18 00 00 00 D4 05 00 00
00003AD0  14 E5 FF FF 34 00 00 00 00 44 0E 50 44 81 0E 64
00003AE0  C1 44 0E 00 18 00 00 00 F0 05 00 00 2C E5 FF FF
00003AF0  40 00 00 00 00 44 0E 50 44 81 0E 70 C1 44 0E 00
```

如图所示，偏移地址为00003AF0，每个文件大小是16个byte

2.4 接下来，请你对照 `vmlinux.lds` 和 `System.map`，查看起始和末尾符号（`_skernel` 和 `_ebss`）对应的地址之差是多少？（在这里，我们暂时不考虑 `_ekernel`，因此存在内存对齐的因素）

```
0000000080200000 T _skernel
```

```
0000000080205008 B _ebss
```

差为0x5008

2.5 请你思考：为什么 `Image` 文件的大小会**小于**链接脚本中定义的内核大小？在上面，我们已经知道 `Image` 文件的内存布局和运行时是一致的，理论上它应当符合链接脚本和符号表中的定义。（提示：你需要理解链接脚本中各个段存放的数据类型）

事实上，由于 `.data` 段存放有初始值的变量，这些初始值本身必须被保存在镜像中，而 `.bss` 段存放未初始化的变量，我们只需要在镜像中记录这个段的大小，而不需要存储其实际内容。内核启动时，加载器会为它分配空间并**直接清零**内存。因此，`.bss` 段的大小不会反映在 `Image` 文件中。

2.6 接下来，请你寻找 `vmlinux.lds` 中栈空间在哪里被定义，并指出具体的代码。我们为什么选择在 `.bss` 段开辟栈空间，而不是 `.data` 段呢？

```
    .bss : ALIGN(0x1000) {
        *(.bss.stack)
        . = ALIGN(0x1000);
        _sbss = .;

```

此时的 `sp` 是「OpenSBI 的栈指针」，而非「内核的栈指针」—— 内核需自行初始化栈后，`sp` 才会切换到内核的 `.bss` 栈区域。

3 Task 1：为 `start_kernel()` 准备运行环境

```
la sp, \_sbss //由于不能直接访问OpenSBI的内存空间，所以指向内核栈的起始位置，_sbss就是那个符号
```

效果展示：

成功运行后，可以设置断点，通过逐条执行，进入内核栈，直到到达printk处

info register scause 显示为0，表面没有出错

```
printk.c
12
13 void printk(const char *fmt, ...)
B+>  14 {
15         FILE printk_out = {
16                 .write = printk_sbi_write,
17         };
18

B+>0x80200150 <printk>      addi    sp,sp,-128
   0x80200154 <printk+4>    sd      ra,56(sp)
   0x80200158 <printk+8>    sd      a1,72(sp)
   0x8020015c <printk+12>   sd      a2,80(sp)
   0x80200160 <printk+16>   sd      a3,88(sp)
   0x80200164 <printk+20>   sd      a4,96(sp)
   0x80200168 <printk+24>   sd      a5,104(sp)
   0x8020016c <printk+28>   sd      a6,112(sp)

remote Thread 1.1 (src) In: printk          L14   PC: 0x80200150
(gdb) c
Continuing.

Breakpoint 2, printk (
    fmt=fmt@entry=0x80203000 "Hello, ZJU OS 2025!\n")
    at printk.c:14
(gdb) i r scause
scause         0x0      0
(gdb)
```

4 Task 2：使用 SBI 实现 `printk()`

这个任务要实现sbi_ecall并补全sbi_debug_console_*()

SBI是S级和M级之间的接口，所以sbi_ecall()函数是所有SBI调用的基础。需要使用 C 的内联汇编功能，将参数正确地放入指定的寄存器，然后执行 ecall 指令，最后从寄存器中取回返回值。

补全 sbi_debug_console_*(): 这几个函数是上层接口，它们需要调用我们刚刚实现的 sbi_ecall()，并根据 [RISC-V SBI 规范]() 传递正确的 EID (Extension ID) 和 FID (Function ID)。

```
struct sbiret sbi_ecall(uint64_t eid, uint64_t fid, uint64_t arg0,
      uint64_t arg1, uint64_t arg2, uint64_t arg3,
      uint64_t arg4, uint64_t arg5)
{
  /* Lab1 Task2: 使用内联汇编实现 SBI 调用 */
  struct sbiret ret;

  // 1. 将 C 变量的值加载到指定的寄存器中
  // "r" 表示使用通用寄存器，asm() 会自动选择
  // "a0" 等是寄存器的名字
  register uint64_t a0 asm("a0") = arg0;
  register uint64_t a1 asm("a1") = arg1;
  register uint64_t a2 asm("a2") = arg2;
  register uint64_t a3 asm("a3") = arg3;
  register uint64_t a4 asm("a4") = arg4;
  register uint64_t a5 asm("a5") = arg5;
  register uint64_t a6 asm("a6") = fid;
  register uint64_t a7 asm("a7") = eid;

  asm volatile(
    "ecall"
    // 2. ecall 执行后，返回值会写回到 a0 和 a1 寄存器
    // "+r" 表示这个操作数既是输入也是输出
    : "+r"(a0), "+r"(a1)
    // 3. 将其他参数作为输入操作数
    : "r"(a2), "r"(a3), "r"(a4), "r"(a5), "r"(a6), "r"(a7)
    // 4. 告知编译器，内存可能会被修改
    : "memory"
  );

  // 5. 从寄存器变量中取回返回值
  ret.error = a0;
  ret.value = a1;

  return ret;
}
```

5 CSR寄存器

在qumu中，输入info registers，即可查看以下寄存器值

```
CPU#0
 V      =   0
 pc       0000000080200000
 mhartid  0000000000000000
 mstatus  8000000a00006080
 hstatus  0000000200000000
 vsstatus 0000000a00000000
 mip      0000000000000020
 mie      0000000000000008
 mideleg  0000000000001666
 hideleg  0000000000000000
 medeleg  0000000000f4b509
 hedeleg  0000000000000000
 mtvec    00000000800004f8
 stvec    0000000080200000
 vstvec   0000000000000000
 mepc     0000000080200000
 sepc     0000000000000000
 vsepc    0000000000000000
 mcause   0000000000000002
 scause   0000000000000000
 vscause  0000000000000000
 mtval    0000000032102573
 stval    0000000000000000
 htval    0000000000000000
 mtval2   0000000000000000
 mscratch 0000000080047000
 sscratch 0000000000000000
 satp     0000000000000000
 x0/zero  0000000000000000 x1/ra    0000000080010588 x2/sp
```

mstatus

位偏移（从 0 开始）	位名称	功能描述	当前位值	具体含义
3	MIE	M 模式中断总开关	0	**M 模式中断全局禁用**（即使 `mie` 寄存器使能特定中断，MIE=0 时也无法触发）。
7	SIE	S 模式中断总开关（M 模式控制）	0	**S 模式中断全局禁用**（M 模式通过此位控制 S 模式的中断总开关，当前禁止 S 模式响应任何中断）。
11	MPRV	内存权限检查模式	0	访问内存时，按「当前特权级（M 模式）」检查权限（MPRV=1 时按 `mprvaddr` 指定的特权级检查，用于内核访问用户内存）。
12	SUM	S 模式用户内存访问允许	0	S 模式下**禁止访问用户态内存**（SUM=1 时 S 模式可访问 U 模式内存，用于内核处理用户请求）。
13	MXR	可执行内存读允许	1	允许读取「标记为可执行（X）」的内存页（MXR=0 时只能读可读写（RW）页，当前配置支持代码段读取）。
17	SPP	S 模式异常返回特权级	0	S 模式发生异常后，执行 `sret` 返回时，恢复到 **U 模式**（SPP=1 时返回 S 模式，当前为默认初始值）。
22	MPIE	M 模式中断使能历史	0	记录「进入当前 M 模式前的 MIE 状态」（用于异常处理后恢复中断开关，当前无历史状态）。
26	SPIE	S 模式中断使能历史	0	记录「进入当前 M 模式前的 SIE 状态」（当前无历史状态）。
31	SD	状态寄存器保存深度	1	表示 `mstatus` 寄存器的「中断使能位（MIE/SIE）」和「特权级位（SPP）」需要被保存（用于上下文切换时完整保存状态）。
63	UXL	U 模式地址宽度	1（二进制 `10`）	U 模式支持 **64 位地址**（UXL=0 对应 32 位，1 对应 64 位，当前为 64 位系统默认配置）。

寄存器	十六进制值	具体含义解析
`mip`	`0000000000000020`	- 位 5（SEIE）：`1` → **S 模式外部中断已触发并挂起**（需注意：S 模式中断是否处理，还需看 `mideleg` 是否委托给 S 模式，以及 `sstatus.SIE` 是否使能）； - 其他位：`0` → 无 M 模式中断挂起，无 S 模式定时器 / 软件中断挂起； - 核心结论：当前有一个 S 模式外部中断等待处理，但因 `mstatus.SIE=0`（S 模式中断总开关禁用），暂时无法响应。
`mcause`	`0000000000000002`	- 最高位（位 63）=0 → **异常（非中断）**； - 低 63 位 = 2 → 原因码为 2，对应「非法指令异常」（RISC-V 规范定义：原因码 2 表示 CPU 执行了未定义或不允许的指令，如 S 模式执行 M 模式专属指令 `csrr mstatus`）； - 结合 `mepc=0x80200000` 推测：上一次在 `0x80200000` 地址执行了非法指令，触发 M 模式异常。
`mtval`	`0000000032102573`	结合 `mcause=2`（非法指令异常），`mtval` 存储的是「触发异常的非法指令机器码」（`0x32102573`）；异常处理程序可通过 `mtval` 定位具体是哪条非法指令导致的异常（若为内存访问异常，`mtval` 会存储错误的内存地址）。
`mepc`	`0000000080200000`	值为 `0x80200000` → 表示 **上一次 M 模式异常发生时，正在执行的指令地址是 `0x80200000`**（此地址是 S 模式异常入口 `stvec` 的值，推测异常是「S 模式触发异常后，因某种原因升级到 M 模式处理」，如 S 模式未处理的严重异常）；执行 `mret` 指令时，CPU 会跳回 `mepc` 地址继续执行。
mie	0000000000000008	`0x8` 对应二进制 `1000`，即 **bit 3 置 1**。 RISC-V 中 `mie` 的 bit 3 是 `MEIE`（M 模式外部中断使能位）—— 表示「仅允许 M 模式外部中断（如 UART 中断、键盘中断）被触发」，其他类型中断（如定时器、软件中断）均被禁用。

S 模式 CSR 寄存器（Supervisor Mode）

寄存器名称	十六进制值	具体含义解析
sstatus	0000000000000000	**S 模式状态寄存器**：记录 S 模式的运行状态，当前值全为 0，说明： 1. 第 1 位（SIE）= 0：S 模式中断总开关关闭； 2. 第 8 位（SPP）= 0：上一次特权级为 U 模式（当前未进入过 S 模式中断 / 异常）； 3. 其他位（如 SUM、MXR）= 0：未开启「 supervisor 访问用户内存」「访问可执行内存视为读」等功能（内核未初始化 S 模式配置）。
sip	0000000000000000	**S 模式中断挂起寄存器**：记录「已发生但未处理」的 S 模式中断，当前值全为 0：无 S 模式软件中断（SSIP）、外部中断（SEIP）、定时器中断（STIP）挂起（即使 M 模式有定时器中断挂起，因未委托或 SIE 关闭，未同步到 S 模式）。
sie	0000000000000000	**S 模式中断使能寄存器**：控制「哪些 S 模式中断允许触发」，当前值全为 0： 禁止 S 模式软件中断（SSIE）、外部中断（SEIE）、定时器中断（STIE）触发（内核未配置 S 模式中断使能）。
stvec	0000000080200000	**S 模式异常向量基地址寄存器**：指定 S 模式异常发生时的跳转地址，格式同 `mtvec`： 1. 低 2 位 = 0（直接模式）：所有 S 模式异常跳转到基地址； 2. 基地址 = `0x80200000`：S 模式异常处理程序的入口地址（内核 `_start` 附近，内核初始化时配置，用于处理委托来的异常）。
scause	0000000000000000	**S 模式异常原因寄存器**：记录 S 模式异常的类型和原因，当前值全为 0： **尚未发生过任何 S 模式异常**（当前异常发生在 M 模式，未委托给 S 模式）。
sepc	0000000000000000	**S 模式异常程序计数器**：记录 S 模式异常发生前的指令地址，当前值全为 0： 因未发生过 S 模式异常，故无地址可记录。
stval	0000000000000000	**S 模式异常值寄存器**：补充 S 模式异常的具体信息，当前值全为 0： 因未发生过 S 模式异常，故无异常相关值可存储。
sscratch	0000000000000000	**S 模式暂存寄存器**：S 模式异常处理程序用于暂存通用寄存器值的临时空间，当前值全为 0： 内核尚未初始化 S 模式异常处理流程，故未配置该寄存器（后续内核会将其设置为 S 模式栈地址）。
satp	0000000000000000	**S 模式地址转换和保护寄存器**：控制 S 模式虚拟内存的启用和页表基地址，当前值全为 0： 1. 低 4 位（MODE）= 0：表示 S 模式未启用虚拟内存（使用物理地址访问内存）； 2. 高 60 位（PPN）= 0：无页表基地址（虚拟内存未启用时无效）。

5.1 在 `_start()` 打断点，然后单步调试，进入 `start_kernel()` 后程序能顺利跳转到 `printk()` 吗？如果不顺利，检查合适的 CSR 寄存器，看看发生了什么异常？

如图所示，不能顺利跳转，因为位于OpenSBI受保护的内存空间，所以无法访问。

scause显示为0x7，即内存泄露。

```
head.S
        8     # la sp, _sbss //由于不能直接访问OpenSBI的内存空间
        9     /* Lab1 Task3 */
       10
B+>    11     tail start_kernel
       12
       13     .section .bss.stack
       14     .space PGSIZE

B+>0x80200000 <_stext>            j       0x8020000c <start_ker
   0x80200004 <_traps>            jal     0x802007b4 <trap_hand
   0x80200008 <_traps+4>          sret
   0x8020000c <start_kernel>      addi    sp,sp,-112
   0x80200010 <start_kernel+4>    sd      ra,104(sp)
   0x80200014 <start_kernel+8>    sd      s0,96(sp)
   0x80200018 <start_kernel+12>   auipc   a0,0x3
   0x8020001c <start_kernel+16>   addi    a0,a0,-24
   0x80200020 <start_kernel+20>   jal     0x80200148 <printk>
remote Thread 1.1 (src) In: _stext          L11   PC: 0x80200000
sp             0x80046c10       0x80046c10
(gdb) si
start_kernel () at main.c:8
(gdb) si
(gdb) si

Breakpoint 1, _stext () at head.S:11
(gdb) i r scause
scause         0x7      7
(gdb) i r mcause
mcause         0x7      7
```

删掉在 `_start()` 打的断点，在 `printk()` 入口处打断点，然后继续运行（continue），你会发现最终能够到达 `printk()` 停下来，这是为什么？

如图所示，此时位置已经不位于OpenSBI的内存范围内，所以可以访问。

```
remote Thread 1.1 (src) In: printk          L14   PC: 0x80200148
(gdb) delete 1
(gdb) break printk
Breakpoint 2 at 0x80200148: file printk.c, line 14.
(gdb) c
Continuing.

Breakpoint 2, printk (
    fmt=fmt@entry=0x80203000 "Hello, ZJU OS 2025!\n")
    at printk.c:14
(gdb) i r sp
sp             0x7fffff60       0x7fffff60
(gdb)
```

5.2 Task 3: Trap Handler

trap概念分析：

在 RISC-V 架构中，Trap 是指任何导致程序控制流意外改变的事件，分为**中断 (Interrupt)** 和**异常 (Exception)** 两类。当中断或异常发生时，CPU 会执行以下原子操作：

1. 停止当前指令流。
2. 将 Trap 的原因代码写入 scause 寄存器。
3. 将被打断的指令地址（返回地址）写入 sepc 寄存器。
4. 将一些与 Trap 相关的信息写入 stval 寄存器。
5. 保存当前的特权级，并进入更高一级的特权模式（例如从 S-Mode 进入 M-Mode，或在 S-Mode 内部处理）。
6. 跳转到 stvec 寄存器指定的地址开始执行 Trap 处理程序。

本次实验主要涉及以下几个关键的 CSR：

- stvec (Supervisor Trap Vector): 指向 S-Mode Trap 处理程序的基地址。
- sstatus (Supervisor Status): 包含 CPU 的全局状态，其中的 SIE 位是 S-Mode 中断的全局总开关。
- sie (Supervisor Interrupt Enable): 控制哪些 S-Mode 中断源被使能（例如软件中断、时钟中断）。
- sip (Supervisor Interrupt Pending): 显示当前有哪些 S-Mode 中断正在等待处理。

- scause, sepc, stval: 分别用于记录 Trap 的原因、返回地址和附加信息。

5.2.1 补全 csr_read, csr_write, 和 csr_set 这三个宏

这些宏是操作系统与硬件底层交互的关键，用于读取和修改处理器的状态，例如启用/禁用中断、设置 Trap 处理地址等。

1. **csr_read(csr)**: 从指定的 CSR (csr) 中读取值。**汇编指令**: csrr rd, csr (Control and Status Register Read)。这个指令会把 csr 寄存器的值读到通用寄存器 rd 中。**内联汇编**: 把 csr 的值输出到一个 C 变量中。
2. **csr_write(csr, val)**: 将一个值 (val) 写入到指定的 CSR (csr) 中。**汇编指令**: csrw csr, rs (Control and Status Register Write)。这个指令会把通用寄存器 rs 的值写入到 csr 寄存器中。**内联汇编**: 把一个 C 变量的值作为输入，写入到 csr。
3. **csr_set(csr, val)**: 对指定的 CSR (csr) 进行“置位”操作，也就是 csr = csr | val。这是一个原子操作。**汇编指令**: csrs csr, rs (Control and Status Register Read and Set)。这个指令会原子地将 csr 寄存器中对应 rs 寄存器中为 1 的位设置为 1。**内联汇编**: 和 csr_write 类似，将一个包含“位掩码”的 C 变量作为输入。

5.2.2 在 `start_kernel()` 中

1. **打印初始状态**: 我们首先使用 csr_read 宏打印出 sstatus、sie 和 sip 的当前值。这对于调试非常重要，可以让你了解在你的操作之前，这些寄存器的状态是什么。
2. **csr_write(sie, SIE_SSIE)**:sie (Supervisor Interrupt Enable) 寄存器是一个“位掩码”，每一位对应一种中断源。SIE_SSIE (Supervisor Software Interrupt Enable) 是 sie 的第 1 位。我们使用 csr_write 将 sie 的值**覆盖**为 SIE_SSIE。这确保了只有 S 模式软件中断被使能，而其他所有 S 模式中断（如时钟中断、外部中断）都被禁用，从而隔离了实验环境。
3. **csr_set(sstatus, SSTATUS_SIE)**:sstatus (Supervisor Status) 寄存器包含了 CPU 的多种状态，其中 SIE (Supervisor Interrupt Enable) 位是 S 模式中断的总开关。我们使用 csr_set 而不是 csr_write。csr_set 执行的是原子性的 read-modify-write 操作（即 sstatus |= SSTATUS_SIE），它只将 SIE 位置 1，而**不会改变** sstatus 中的其他位（例如 SPP 位，它记录了进入 trap 前的特权级）。这在操作系统中是至关重要的。
4. **csr_set(sip, SIP_SSIP)**:sip (Supervisor Interrupt Pending) 寄存器反映了当前有哪些中断正在等待处理。SIP_SSIP 是 sip 的第 1 位，表示 S 模式软件中断正在 pending。通过设置这一位，我们手动“制造”了一个待处理的软件中断。由于此时 sie.SSIE 和 sstatus.SIE 都已经被使能，CPU 会立即响应这个中断，暂停 start_kernel 的执行，并跳转到 stvec 寄存器所指向的中断处理程序。

代码如下：

```
noreturn void start_kernel(void)
{
  printk("Hello, ZJU OS 2025!\n");
  printk("SBI spec version: %#08lx\n", sbi_get_spec_version().value);
  printk("SBI impl id: %#lx\n", sbi_get_impl_id().value);
  printk("SBI impl version: %#08lx\n", sbi_get_impl_version().value);
  printk("SBI machine vendor id: %lu\n", sbi_get_mvendorid().value);
  printk("SBI machine arch id: %lu\n", sbi_get_marchid().value);
  printk("SBI machine imp id: %lu\n", sbi_get_mimpid().value);
/* Lab1 Task3: Trigger a software interrupt */
printk("\n--- Lab1 Task3: Trap Handler ---\n");

// 1. 打印 sstatus、sie 和 sip 的初始值，以供观察
printk("Initial CSR state:\n");
printk("sstatus: %#018lx\n", csr_read(sstatus));
```

```
printk("sie:     %#018lx\n", csr_read(sie));
printk("sip:     %#018lx\n", csr_read(sip));

// 2. 将 sie 设置为只使能 S 模式软件中断 (SSIE)
// 使用 csr_write 会覆盖 sie 的原值，确保只有软件中断被使能。
printk("Enabling Supervisor Software Interrupt in sie...\n");
csr_write(sie, SIE_SSIE);
printk("sie after write: %#018lx\n", csr_read(sie));

// 3. 在 sstatus 中全局使能 S 模式中断 (SIE)
// 使用 csr_set 执行原子"或"操作，只设置 SIE 位而不影响 sstatus 的其他状态。
printk("Enabling S-mode interrupts globally in sstatus...\n");
csr_set(sstatus, SSTATUS_SIE);
printk("sstatus after set: %#018lx\n", csr_read(sstatus));

// 4. 将 sip 的 SSIP 位置 1，以触发一个 S 模式软件中断
// 根据规范，S 模式通常不能直接写入 sip，这通常由 M 模式或硬件完成。
// 在本实验环境中，我们假定可以通过 SBI 调用或特定机制来设置此位。
// 一旦 SSIP 位被置位，由于 sie 和 sstatus 已经配置好，CPU 将立即陷入中断处理程序。
printk("Setting sip.SSIP to trigger a software interrupt...\n");
// 这行代码执行后，预计会立即发生中断，程序流将跳转到 stvec 指定的地址。
csr_set(sip, SIP_SSIP);

// 如果你的中断处理程序正确返回，下面的代码才会继续执行。
// 在许多情况下，中断处理后可能会进入任务调度或死循环，因此你可能看不到下面的输出。
printk("...Software interrupt triggered.\n");
printk("--- Lab1 Task3 Finished ---\n\n");
```

5.2.3 在 `head.S` 中，写入合适的 CSR 寄存器，将中断处理程序设置为 `_traps`

简单来说，就是在内核启动的最早期阶段设置 Trap（中断和异常）处理程序的入口地址。

1. **目标寄存器**: 在 RISC-V S-Mode (Supervisor Mode) 下，控制 Trap 入口地址的 CSR 是 stvec (Supervisor Trap Vector Base Address Register)。
2. **目标地址**: 你的任务要求将中断处理程序设置为 *traps*。traps 是一个符号（label），代表了中断处理程序的起始地址。
3. **汇编指令**:la t0, _traps: la (Load Address) 是一个伪指令，它会将 _traps 的地址加载到临时寄存器 t0 中。csrw stvec, t0: csrw (Control and Status Register Write) 指令会将 t0 寄存器中的值（也就是 _traps 的地址）写入到 stvec 寄存器中。

代码如下所示

```
/* Lab1 Task3: Set stvec to point to the trap handler */
// 1. 将中断处理程序 _traps 的地址加载到临时寄存器 t0。
la t0, _traps
// 2. 将 t0 中的地址写入 stvec 寄存器。
//    从此以后，在 S 模式下发生任何中断或异常，
//    CPU 都会自动跳转到 _traps 处开始执行。
csrw stvec, t0

// 跳转到 C 语言的入口函数 start_kernel，并且不再返回。
tail start_kernel
```

```
// 为内核初始栈分配空间
.section .bss.stack
.space PGSIZE
```

5.2.4 在 `entry.S` 中，补全 `_traps`

此处结合思考题进行分析

保存的原因：Trap 发生时，CPU 正在执行的程序被强制打断。这个被打断的程序对 Trap 的发生是“无感的”。为了让它在 Trap 处理结束后能无缝地继续执行，Trap Handler **必须保证不能修改任何可能被原程序使用的寄存器**。
RISC-V 调用约定区分了“调用者保存 (caller-saved)”和“被调用者保存 (callee-saved)”寄存器。但在 Trap 的场景下，被打断的程序是“调用者”，但它并没有机会保存任何东西。因此，作为“被调用者”的 _traps 必须承担起**保存所有通用寄存器**的责任。

trap_handler() 需要的参数：

1. **scause**: Trap 的原因。是中断还是异常？具体是哪一种？
2. **sepc**: Trap 发生时的指令地址 (Program Counter)。处理完成后，CPU 需要知道该返回哪里继续执行。
3. **stval**: 与 Trap 相关的附加信息。例如，如果是缺页异常，stval 会存放导致异常的地址。
4. **指向 Trap 帧的指针**: C 代码可能需要读取或修改被保存的寄存器。例如，处理系统调用时，需要从 a0-a7 中读取参数；返回时，可能需要将返回值写入 a0。将 Trap 帧的地址（也就是 sp）作为参数传递进去，C 代码就可以通过一个结构体指针来访问所有寄存器。

此处代码较长，就不另贴了

5.2.5 完成 trap.c 中的中断处理部分

将前面所有设置（stvec、_traps 汇编、sie/sstatus 配置）联系在一起的最后一步。

1. **trap_handler(scause, ...)**: 这个 C 函数是所有中断和异常的入口。它的首要任务是**分发 (dispatch)**。它需要检查 scause 寄存器的值，来判断 Trap 的具体原因。
2. **case SCAUSE_SSI**: 在 start_kernel 中，我们触发的是 S 模式软件中断 (Supervisor Software Interrupt)。根据 trap.c 中提供的宏，这个中断对应的 scause 值是 SCAUSE_SSI。因此，我们需要在 switch 语句中捕获这个 case。
3. **clear_ssip()**: 这是 S 模式软件中断的具体处理函数。**核心任务**: 一个中断处理函数最重要的职责之一，就是在处理完中断后**清除中断挂起位 (pending bit)**。否则，当中断处理程序通过 sret 返回后，CPU 会发现中断仍然是挂起状态，从而立即再次进入中断处理程序，导致无限循环，系统卡死。**如何清除**: S 模式软件中断的挂起位是 sip (Supervisor Interrupt Pending) 寄存器中的 SSIP 位。我们需要将这一位清零。**原子操作**: 清除 CSR 中的特定位，最高效、最安全的方式是使用 csrc (CSR Clear) 指令，它执行原子性的 read-modify-write 操作 (csr &= ~mask)。

5.3 动手做

阅读 OpenSBI 源码，了解 OpenSBI 是如何实现 `sbi_set_timer()` 函数的：

- `lib/sbi/sbi_ecall_time.c` 的 `sbi_ecall_set_timer()` 函数
- `lib/sbi/sbi_timer.c` 的 `sbi_timer_event_start()` 函数

请你指出：

- 当平台支持 SSTC 扩展时，OpenSBI 会如何设置定时器？

- 当平台不支持 SSTC 扩展时，OpenSBI 如何进行定时器多路复用?

首先，S-Mode OS 内核调用 sbi_set_timer()，这会触发一个 ecall 指令，陷入到 M-Mode 的 OpenSBI。

1. **ECALL 入口 (sbi_ecall_time.c)**:OpenSBI 的 ECALL 分发器会根据 a7 寄存器中的扩展 ID (SBI_EXT_TIME) 找到 ecall_time 这个扩展。然后调用其 handle 函数，即 sbi_ecall_time_handler()。该函数检查 a6 寄存器中的函数 ID，发现是 SBI_EXT_TIME_SET_TIMER。它从 a0 (和 a1，对于 32 位系统) 中提取出 S-Mode 期望的下一个定时器事件的时间点 next_event。最后，它调用核心函数 sbi_timer_event_start(next_event) 来完成实际的定时器设置。
2. **核心处理逻辑 (sbi_timer.c)**:
   所有的关键逻辑都在 sbi_timer_event_start() 函数中。这个函数根据 CPU 是否支持 SSTC 扩展，采取了两种截然不同的策略。

在支持 SSTC 的情况下，OpenSBI 的 sbi_set_timer() 只是一个简单的“传话筒”，它将 S-Mode 的请求直接写入专用的硬件寄存器 stimecmp。**M-Mode 完全不参与后续的中断生成和传递过程**，这是一种高效且解耦的实现。

在不支持 SSTC 的情况下，OpenSBI 扮演了一个**中间人**的角色。它利用唯一的 M-Mode 物理定时器，先为 S-Mode 安排一次 M-Mode 中断，在 M-Mode 中断处理程序中，再手动**模拟/伪造**一次 S-Mode 中断。通过这种“捕获再转发”的机制，OpenSBI 实现了用一个物理定时器服务于 M-Mode 和 S-Mode 的**多路复用**。

5.4 Task 4:开启并处理 S 模式时钟中断

核心目标是基于已建立的 Trap 机制，为内核引入时间管理能力。这需要实现一个完整、可控的 S-Mode 时钟中断处理流程，并提供一个符合 POSIX 标准的时间获取接口。

围绕 RISC-V 的时间相关 CSR 和机制展开：

1. **time CSR**: 一个 64 位的硬件计数器，以恒定的频率（在 QEMU virt 平台下为 10MHz）自增。它是系统中最原始、最精确的时间基准。由于其 64 位的宽度，在可预见的未来它不会发生回绕。

2. **stimecmp CSR**: S-Mode 的时钟比较寄存器。这是一个由软件设置的“闹钟”。当 time 寄存器的值大于等于 stimecmp 的值时，硬件会自动在 sip 寄存器中置位 STIP (Supervisor Timer Interrupt Pending) 位，从而触发时钟中断。

3. **sbi_set_timer(value)**: OpenSBI 提供的标准 SBI 调用，其作用就是将 value 写入底层的 stimecmp 寄存器。

4. **POSIX clock() 和 CLOCKS_PER_SEC**: clock() 函数应返回程序启动以来的“时钟滴答数”，而 CLOCKS_PER_SEC 定义了每秒钟有多少个滴答。POSIX 标准要求 CLOCKS_PER_SEC 为 1,000,000 (1MHz)，这意味着 clock() 函数返回值的频率必须是 1MHz。

首先，开启并捕获到时钟中断。
实现思路:要触发时钟中断，必须同时满足两个条件：中断源被使能，且中断事件已发生。
使能中断源: 通过修改 sie (Supervisor Interrupt Enable) 寄存器，将 STIE (Supervisor Timer Interrupt Enable) 位置 1，使内核做好接收时钟中断的准备。
触发中断事件: 调用 sbi_set_timer() 设置一个过去的 stimecmp 值。最简单的方法是 sbi_set_timer(0)。因为内核启动后 time 的值必然大于 0，time >= stimecmp 的条件会立即满足，从而立刻触发一次中断。

实现过程:
在 start_kernel() 中，将 csr_write(sie, SIE_SSIE) 修改为 csr_write(sie, SIE_SSIE | SIE_STIE)，同时使能了软件中断和时钟中断。
在代码的Lab1 Task4部分，添加 sbi_set_timer(0); 调用。

其次，为了解决中断风暴问题，我们需要在每次中断处理中，动态地更新 stimecmp，将其设置为一个未来的时间点。
实现思路:
捕获中断: 在 trap_handler 的 switch 语句中，增加对 SCAUSE_STI (Supervisor Timer Interrupt) 的处理分支。
重新“预约”: 在该分支中，调用一个新函数 clock_set_next_event()。此函数的核心逻辑是:
a. 读取当前 time 寄存器的值。
b. 在当前时间的基础上，加上一个固定的时间间隔（例如 1 秒对应的计数值 10,000,000），计算出下一次中断的目标时间点。
c. 调用 sbi_set_timer() 将这个未来的目标时间点写入 stimecmp。

实现过程:
在 trap_handler 中添加 case SCAUSE_STI:，并在其中调用 clock_set_next_event()。实现 clock_set_next_event() 函数，其代码核心为:

```
uint64_t current_time = csr_read(time);
uint64_t next_event_time = current_time + TIMER_FREQ;
// TIMER_FREQ is 10MHz sbi_set_timer(next_event_time);
```

最后是提供一个标准化的时间接口，将底层的硬件时间转换为符合 POSIX 规范的“时钟滴答”。
实现思路:
clock() 的本质是一个单位转换问题。硬件 time CSR 以 10MHz 的频率计数，而 POSIX clock() 需要返回 1MHz 的计数值。因此，我们只需要读取硬件时间，并将其值除以 10 即可。

实现过程:
在 clock.c 的 clock() 函数中，使用内联汇编 asm volatile("csrr %0, time" : "=r"(var)); 直接读取 time CSR 的值。选择直接使用汇编是为了让该文件不依赖于项目中的其他头文件，具有更好的模块化。将读取到的 64 位硬件计数值除以 10，并将结果返回。

```
uint64_t hardware_time;
asm volatile("csrr %0, time" : "=r"(hardware_time));
return hardware_time / 10;`
```